Learn
Practice
Improve

Python
Methods and Objects

Objects have data, methods give them life!

- Understand
- Write Code
- Solve Problems
- Build Skills

Same Concept New Possibilities

1. What is an Object?

- An object is an instance of a class.
- It has its own data (attributes) and can use methods to perform actions.
- You can create multiple objects from the same class, each with different data.

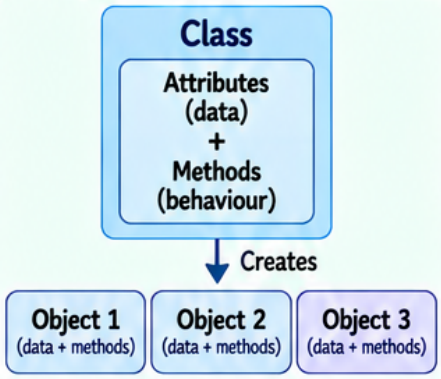
"An object is a real-world entity in your program."

2. What are Methods?

- A method is a function defined inside a class.
- It operates on the object's data (attributes).
- Methods define the behaviour of an object.
- They are called using the object name and dot (.) operator.

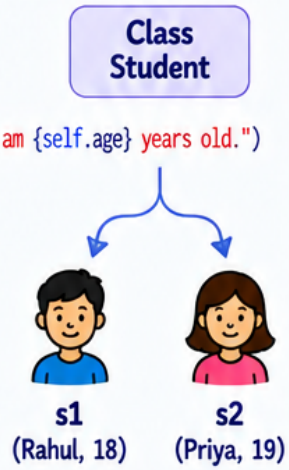
"Methods tell an object what to do."

3. Relationship



4. Example: Class, Objects and Methods

```
1  class Student:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6      def introduce(self):
7          print(f"Hi, I am {self.name} and I am {self.age} years old.")
8
9      def greet(self):
10         print("Keep Learning!")
11
12 # Creating objects
13 s1 = Student("Rahul", 18)
14 s2 = Student("Priya", 19)
15
16 # Calling methods using objects
17 s1.introduce()
18 s1.greet()
19 s2.introduce()
20 s2.greet()
```



5. Output

```
Hi, I am Rahul and I am 18 years old.
Keep Learning!
Hi, I am Priya and I am 19 years old.
Keep Learning!
```

6. How It Works?

1. We create objects (s1, s2) from the class Student.
2. Each object has its own data (name, age).
3. We call methods using the object name (s1.introduce()).
4. The method accesses the object's data using self.
5. Each object can call the same method with different data.

7. Key Points

- An object is created from a class.
- Methods are functions inside a class.
- Methods operate on object data (attributes).
- Call methods using object_name.method_name().
- Multiple objects can use the same method with different data.
- self refers to the current object.

8. Real Life Analogy

A class is like a mobile phone design.
Objects are individual phones (e.g., your phone, friend's phone).
Methods are the functions you can use (call(), message(), camera(), etc.).
Each phone (object) has its own data (contact list, storage, settings).

"Same methods, different objects, different stories." — By Satyarthi

Keep Learning | Keep Practicing | Keep Growing

Python → A Brighter Tomorrow

Python
Methods and Objects
Solutions (Set 1)

Practice
Code
Improve
Grow

Small
Steps
Big
Results

Q1 Student Class with Methods – Answer

Program Code:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(f"Name  : {self.name}")
        print(f"Marks : {self.marks}")

    def is_passed(self):
        return self.marks >= 40

# Create two objects
s1 = Student("Rahul", 85)
s2 = Student("Priya", 32)

# Call methods for s1
s1.display()
if s1.is_passed():
    print("Result: Passed")
else:
    print("Result: Failed")

# Call methods for s2
s2.display()
if s2.is_passed():
    print("Result: Passed")
else:
    print("Result: Failed")
```

Output:

```
Name  : Rahul
Marks : 85
Result: Passed

Name  : Priya
Marks : 32
Result: Failed
```

Explanation:

- We created a class Student with attributes name and marks.
- The display() method shows the student's details.
- The is_passed() method checks if marks are 40 or more.
- We created two objects (s1 and s2) with different values and called both methods for each object.

"Objects make your code more organized and reusable."

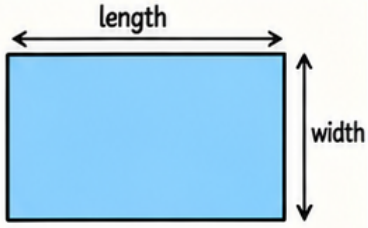
Q2 Rectangle Class with Methods – Answer

Program Code:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

# Create two objects
r1 = Rectangle(5, 3)
r2 = Rectangle(10, 4)

# Display results for r1
print("Rectangle 1:")
print("Area      :", r1.area())
print("Perimeter :", r2.perimeter())

# Display results for r2
print("\nRectangle 2:")
print("Area      :", r2.area())
print("Perimeter :", r2.perimeter())
```

Output:

```
Rectangle 1:
Area      : 15
Perimeter : 16

Rectangle 2:
Area      : 40
Perimeter : 28
```



Explanation:

- We created a class Rectangle with attributes length and width.
- The area() method returns the area (length × width).
- The perimeter() method returns the perimeter (2 × (length + width)).
- We created two objects (r1 and r2) with different values and calculated the area and perimeter for each.

"Use classes and methods to turn simple ideas into powerful solutions."

Keep Learning | Keep Practicing | Keep Improving

— By Satyarthi

3 Multilevel Inheritance

Inheritance in multiple levels (chain)

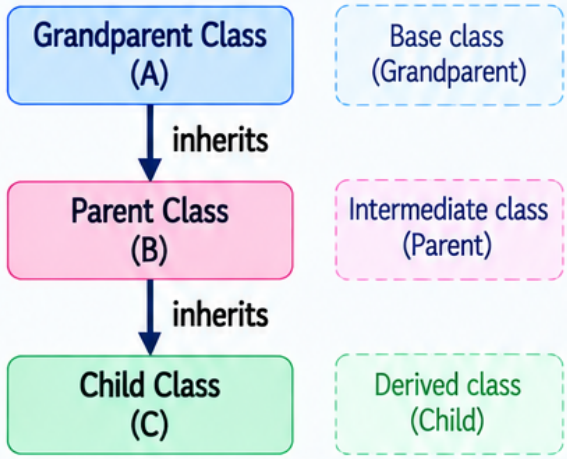
What is Multilevel Inheritance?

- In multilevel inheritance, a class is derived from a child class, and that child class is derived from another parent class.
- This forms a chain of inheritance (more than one level).
- The last child class inherits the properties and methods from all the parent classes in the chain.

> *"Inheritance flows in a chain across multiple levels."*

Learn Practice Apply Grow

Structure



Real Life Analogy

Grandfather (Grandparent) — Common values (e.g., family name, family property)

Father (Parent) — Inherits and adds new things (e.g., education, skills)

Son (Child) — Inherits everything and adds new things (e.g., hobbies, career)

"Each generation gets the values from the previous one and adds something new."

Example Program

```
class Grandparent:
    def show_grandparent(self):
        print("This is Grandparent class")

class Parent(Grandparent): # inherits Grandparent
    def show_parent(self):
        print("This is Parent class")

class Child(Parent): # inherits Parent
    def show_child(self):
        print("This is Child class")

# Creating object of Child class
c = Child()
c.show_grandparent()   # from Grandparent
c.show_parent()        # from Parent
c.show_child()         # from Child
```

Output

```
This is Grandparent class
This is Parent class
This is Child class
```

Key Points

- A class is derived from a child class (chain of inheritance).
- The child class inherits attributes and methods from all the parent classes in the chain.
- Helps in code reusability and reduces redundancy.
- Useful when you have a clear hierarchy.
- Example: Grandparent → Parent → Child.
- Python supports multilevel inheritance.

"Build on what you have, go further with inheritance."

— By Satyarthi

2 Multiple Inheritance in Python

A class inherits from more than one parent class

What is Multiple Inheritance?

- In multiple inheritance, a class inherits from two or more parent classes.
- The child class gets all the attributes and methods from all the parent classes.
- It allows combining features from different classes into a single class.
- Useful when we need to integrate functionalities from multiple sources.

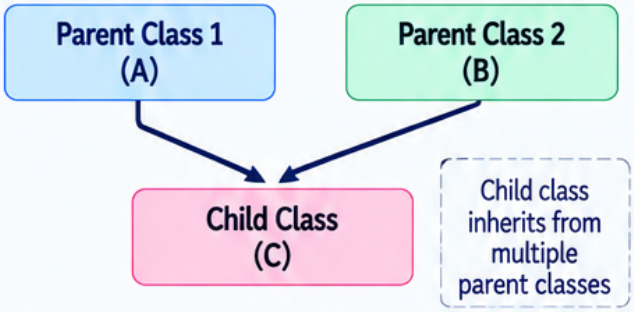
Syntax

```
class Parent1:
    pass

class Parent2:
    pass

class Child(Parent1, Parent2):
    pass
```

Learn
Practice
Apply
Grow

> *Combine strengths, create better code.*

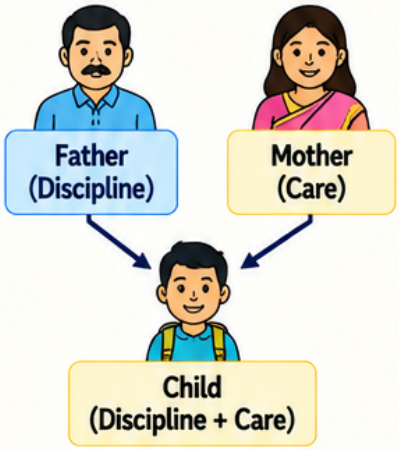
Structure



Example Program

```
class Father:
    def father_property(self):
        print("House from Father")

class Mother:
    def mother_property(self):
        print("Savings from Mother")

class Child(Father, Mother):  # Multiple inheritance
    def show(self):
        print("Child has both properties")

# Creating object of Child class
c = Child()
c.father_property()  # from Father class
c.mother_property()  # from Mother class
c.show()
```

Real Life Analogy



A child inherits qualities from both father and mother, just like a class inherits features from multiple parent classes.

Key Points

- A class inherits from two or more parent classes.
- Child class gets all attributes and methods from all parents.
- Helps in code reusability.
- Useful for combining features from different classes.
- Syntax: `class Child(Parent1, Parent2)`.
- Python supports multiple inheritance.
- Be careful with method name conflicts (MRO – Method Resolution Order).

Output

```
House from Father
Savings from Mother
Child has both properties
```

"More than one source, more possibilities."

— By Satyarthi

Python - Multiple Inheritance
Practice Question

Learn
Practice
Apply
Grow

Q1 **Consider the following Python code:**

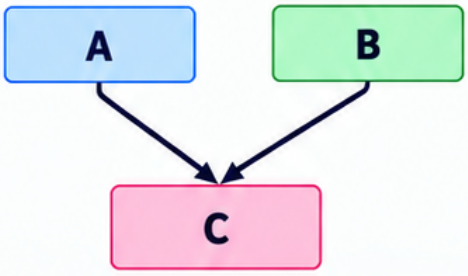
```
class A:
    def feature_a(self):
        print("A")

class B:
    def feature_b(self):
        print("B")

class C(A, B):
    def feature_c(self):
        print("C")
```

? Which of the following diagrams correctly represents the inheritance structure of the above code?

- **A** C inherits from A and B (Multiple Inheritance)
- **B** A inherits from B and C
- **C** B inherits from A and C
- **D** C inherits from B which inherits from A

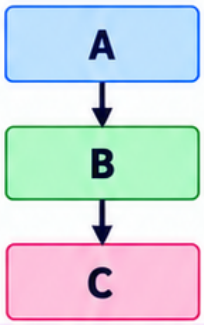
Options (Inheritance Diagrams)

A



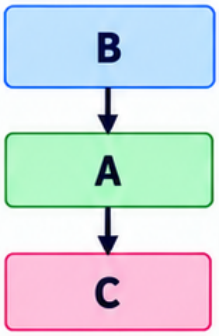
C inherits from A and B (Multiple Inheritance)

B



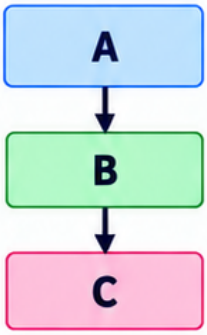
A inherits from B and C

C



B inherits from A and C

D



C inherits from B which inherits from A

"Good questions build strong concepts."

— By Satyarthi

Multilevel Inheritance in Python

A chain of inheritance (one parent, one child, and so on)

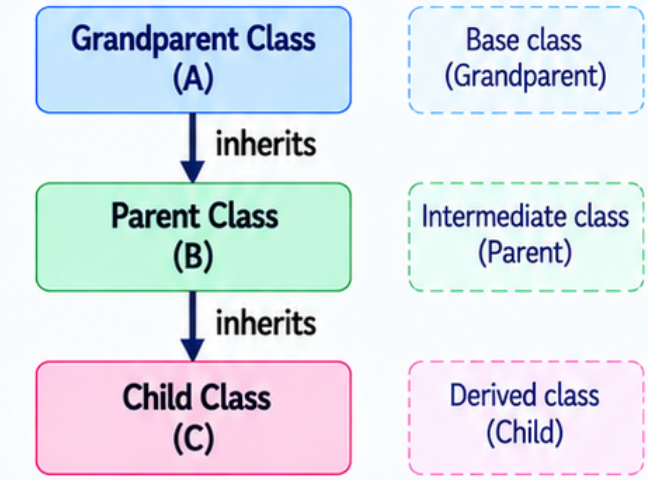
Learn
Practice
Apply
Grow

What is Multilevel Inheritance?

- In multilevel inheritance, a class is derived from a child class, and that child class is derived from another parent class.
- It forms a chain of inheritance (more than one level).
- The last child class inherits the properties and methods from all the parent classes in the chain.
- It helps in code reusability and reduces redundancy.

> *"Knowledge is inherited, but creativity builds the future."*

Class Hierarchy



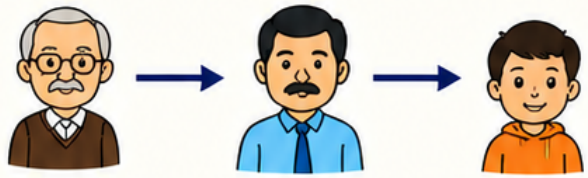
Example Program

```
class A:                    # Grandparent class
    def show_a(self):
        print("This is class A")

class B(A):                 # Parent class (inherits A)
    def show_b(self):
        print("This is class B")

class C(B):                 # Child class (inherits B)
    def show_c(self):
        print("This is class C")

# Creating object of child class
obj = C()
obj.show_a()   # from class A
obj.show_b()   # from class B
obj.show_c()   # from class C
```

Output

```
This is class A
This is class B
This is class C
```

Real Life Analogy



Grandfather (A)	Father (B)	Son (C)
Has family values	Inherits values and adds skills	Inherits values, skills and adds new qualities

Key Points

- A class is derived from a child class (chain).
- The last child class inherits from all previous classes.
- Helps in code reusability.
- Useful when there is a clear hierarchy.
- Example: A → B → C.
- Python supports multilevel inheritance.

"Build on what you inherit, and create something better."

— By Satyarthi

Python Encapsulation

Better Control
More Security
Clean Code

Wrapping data (variables) and methods (functions) together in a single unit (class) and restricting direct access to some of the data is called Encapsulation.

Why Encapsulation?

- Protects data from unauthorized access
- Hides internal details
- Provides better control over data
- Makes the code more secure and maintainable

How it works in Python?

In Python, encapsulation is achieved using access modifiers:

Modifier	Syntax	Meaning
Public	`self.variable`	Accessible from anywhere
Protected	`self._variable`	By convention, avoid direct access (outside class)
Private	`self.__variable`	Not directly accessible (outside class)

Example: Encapsulation in Python

```python
class Student:
    def __init__(self, name, marks):
        self.name = name        # Public variable
        self._roll_no = 101     # Protected variable
        self.__marks = marks    # Private variable

    def show_details(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self._roll_no}")
        print(f"Marks: {self.__marks}")

    def get_marks(self):   # Getter method
        return self.__marks

    def set_marks(self, marks):   # Setter method
        if marks >= 0 and marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks!")

# Creating object and using methods
s = Student("Rahul", 85)
s.show_details()
print("Marks (using getter):", s.get_marks())
s.set_marks(90)
print("Updated Marks:", s.get_marks())
```

Output

```
Name: Rahul
Roll No: 101
Marks: 85
Marks (using getter): 85
Updated Marks: 90
```

Key Points

- Encapsulation is implemented using classes.
- Use private variables (__) to restrict direct access.
- Use getter and setter methods to access or modify private data.
- Helps in data security and code modularity.
- Encapsulation is one of the 4 pillars of OOP (along with Inheritance, Polymorphism, and Abstraction).

Encapsulation = Data Hiding + Controlled Access = Better Code

Python
Public Attributes

Public = Accessible Everywhere

Public attributes are variables (data members) that can be accessed from anywhere – inside the class and outside the class.

What are Public Attributes?

- Public attributes are declared normally inside a class (without any underscore).
- They are accessible from inside the class and outside the class.
- In Python, by default, all attributes are public.
- Useful when we want the data to be freely accessible.

Syntax

No underscore = Public

```
class ClassName:
    def __init__(self):
        self.attribute = value  # Public attribute
```

Public attribute name can be anything (e.g., name, age, marks).

Example: Public Attributes in Python

```
class Student:
    def __init__(self, name, age):
        self.name = name     # Public attribute
        self.age = age       # Public attribute

    def show_info(self):
        print("Name:", self.name)
        print("Age:", self.age)

# Creating an object
s = Student("Rahul", 19)

# Accessing public attributes
# from outside the class
print("Accessing from outside:")
print("Name:", s.name)
print("Age:", s.age)

# Modifying public attribute
s.name = "Amit"
print("\nAfter modification:")
print("Name:", s.name)
```

Output

```
Name: Rahul
Age: 19

Accessing from outside:
Name: Rahul
Age: 19

After modification:
Name: Amit
```

Key Points

1. Declared normally without underscore.
2. Accessible from anywhere (inside and outside the class).
3. Default in Python – all attributes are public.
4. Can be modified from outside the class.
5. Use public attributes when data needs to be freely available.

Public Attribute = No Restriction = Easy Access

Python
Protected Attributes

Single Underscore (_) = Protected

Protected attributes are variables (data members) that are intended to be used inside the class and its subclasses.
They are not meant to be accessed directly from outside the class.

Key Features

- Declared with a single underscore (_).
- Intended for internal use within the class and subclasses.
- Can still be accessed from outside (not truly private).
- Helps in data protection and follows the principle of data hiding.

Syntax

```
class ClassName:
    def __init__(self):
        self._attribute = value
```

The single underscore (_) is a convention to indicate that the attribute is protected.

Real-Life Analogy

A protected attribute is like a room in a house marked **"Family Use Only"**.

It's accessible to family members (inside the class and subclasses), but guests (outside the class) are not supposed to use it directly.

Example 1: Protected Attribute in Python

```
class Employee:
    def __init__(self, name, salary):
        self.name = name        # Public attribute
        self._salary = salary   # Protected attribute

    def show_details(self):
        print(f"Name: {self.name}")
        print(f"Salary: {self._salary}")

# Creating object
emp = Employee("Rohan", 50000)

# Accessing attributes
print(emp.name)      # Public - accessible
print(emp._salary)   # Protected - accessible (but not recommended)
emp.show_details()
```

Output

```
Rohan
50000
Name: Rohan
Salary: 50000
```

Example 2: Access in Subclass (Inheritance)

```
class Manager(Employee):      # Subclass
    def show_salary(self):
        print(f"(From subclass) Salary: {self._salary}")

# Creating object of subclass
m = Manager("Sneha", 75000)
m.show_salary()
```

Output

```
(From subclass)
Salary: 75000
```

Key Points

1. Use single underscoce (_).
2. Accessible inside the class.
3. Accessible in subclasses (inherited classes).
4. Should not be accessed directly from outside the class.
5. Useful for data protection and controlled access.

Protected Attribute = Limited Access = Better Control

Python Abstraction

Hide the Complexity Show only What is Needed

Abstraction is the concept of **hiding** the **internal implementation details** and showing only the essential features (what to do) to the user. It focuses on **"what an object does"** rather than **"how it does it"**.

Key Points

- Hides complex implementation details.
- Shows only the necessary features to the user.
- Implemented using abstract classes and abstract methods (in Python using abc module).
- Helps in reducing complexity and improving code security.
- Allows multiple classes to provide their own implementation (same interface).

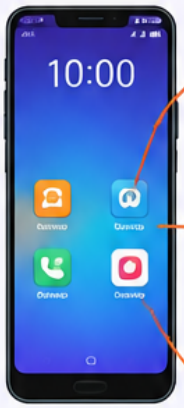
Real-Life Example

Car Driving

- When you drive a car, you use the steering wheel, accelerator and brake.
- You don't need to know how the engine works internally.
- The internal mechanism (engine, fuel injection, etc.) is hidden from you.
- You only interact with essential features like start, accelerate, brake, steer.

This is Abstraction!
You use the functionality without knowing the internal details.

Analogy



You press icons to make calls, click photos, etc.

You don't know the internal code, hardware working, or processing logic.

You just use the features provided.

Abstraction hides the complex backend and gives a simple interface to the user.

Example: Abstraction in Python

```python
from abc import ABC, abstractmethod

# Abstract class
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

# Child classes providing implementation
class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

# Using the classes
dog = Dog()
cat = Cat()
print("Dog sound:", dog.sound())
print("Cat sound:", cat.sound())
```

Output

```
Dog sound: Bark
Cat sound: Meow
```

Explanation

- `Animal` is an abstract class with an abstract method sound().
- Dog and Cat classes provide their own implementation of sound().
- We cannot create an object of the abstract class `Animal` (it will give an error).
- Abstraction ensures that every subclass must implement the required method.

Key Takeaway

Abstraction = Hide the "How" Show only the "What"

"Simplicity for the User, Complexity behind the Scenes."

Python Abstraction

Focus on What to do Not on How it does it!

Abstraction is the concept of **hiding the internal implementation details** and showing only the essential features (**what to do**) to the user.

Key Points

- **Hides complex implementation details.**
- Shows only the necessary features to the user.
- Implemented using **abstract classes** and **abstract methods** (abc module).
- Helps in reducing complexity and improving code security.
- Allows multiple classes to provide their own implementation (same interface).

Real-Life Example

ATM Machine

You use the ATM:

- Insert card
- Enter PIN
- Select option
  - Withdraw
  - Deposit
  - Check Balance

You don't know how the machine internally:

- verifies your PIN
- connects to the bank server
- processes the transaction
- handles security checks

This is Abstraction!
You use the functionality without knowing the internal working.

Analogy

You press buttons to change channel, increase volume, etc.

You don't know the internal code or hardware working inside the remote.

The remote shows only the essential features (buttons), hiding the complex implementation.

Syntax

```
from abc import ABC, abstractmethod

class ClassName(ABC):
    @abstractmethod
    def method_name(self):
        pass
```

ABC → Abstract Base Class
abstractmethod → Decorator to create an abstract method

Example in Python

```
from abc import ABC, abstractmethod

# Abstract class
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

# Child classes providing implementation
class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"
```

Animal is an abstract class with an abstract method sound().

Dog and Cat classes provide their own implementation of sound().

How It Works

1. We define an abstract class (using ABC).
2. We declare abstract methods (using @abstractmethod).
3. Child classes must provide their own implementation of the abstract methods.
4. We cannot create an object of an abstract class (it will give an error).

Benefits of Abstraction

- Reduces complexity for the user.
- Hides unnecessary implementation details.
- Provides a clear and simple interface.
- Improves security and maintainability.
- Widely used in real-world applications (e.g., ATM, TV remote, smartphone apps, vehicles, etc.).

Abstraction = Simplicity for the User

Python Polymorphism

Same Name Different Behaviour!

Polymorphism means **"many forms"**.
It allows the same method name to have different behaviors for different objects.

Key Points

- Same method name, different behavior.
- The same function or operator works with different types of objects.
- Makes code flexible, reusable and easy to maintain.
- Achieved in Python through method overriding (inheritance) and operator overloading.
- Real-life example: a person can have many roles (student, player, friend) but still the same person.

Types of Polymorphism in Python

1. Compile-Time Polymorphism (Method Overloading)

- Same method name with different parameters.
- Python does not support method overloading directly, but we can achieve similar behavior using default arguments or *args.

2. Runtime Polymorphism (Method Overriding)

- Same method name in parent and child classes, but different implementation.
- The method to be executed is decided at runtime based on the object.
- Achieved using inheritance and method overriding.

Examples

1. Method Overloading (Compile-Time)

Python doesn't support method overloading directly, but we can achieve similar behavior using default arguments.

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

calc = Calculator()
print(calc.add(5, 3))     # 8 (two arguments)
print(calc.add(5, 3, 2)) # 10 (three arguments)
```

Explanation:
The same method name add() works with different numbers of arguments, giving different results.

Same method name, different behavior based on the number of arguments.

2. Method Overriding (Runtime)

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

# Creating objects
animals = [Animal(), Dog(), Cat()]

# Same method name, different behavior
for a in animals:
    a.sound()
```

Output:

```
Animal makes a sound
Dog barks
Cat meows
```

The same method sound() behaves differently depending on the object (Animal, Dog, Cat).

Polymorphism = Same Interface, Different Behavior

"One method name, many forms."